



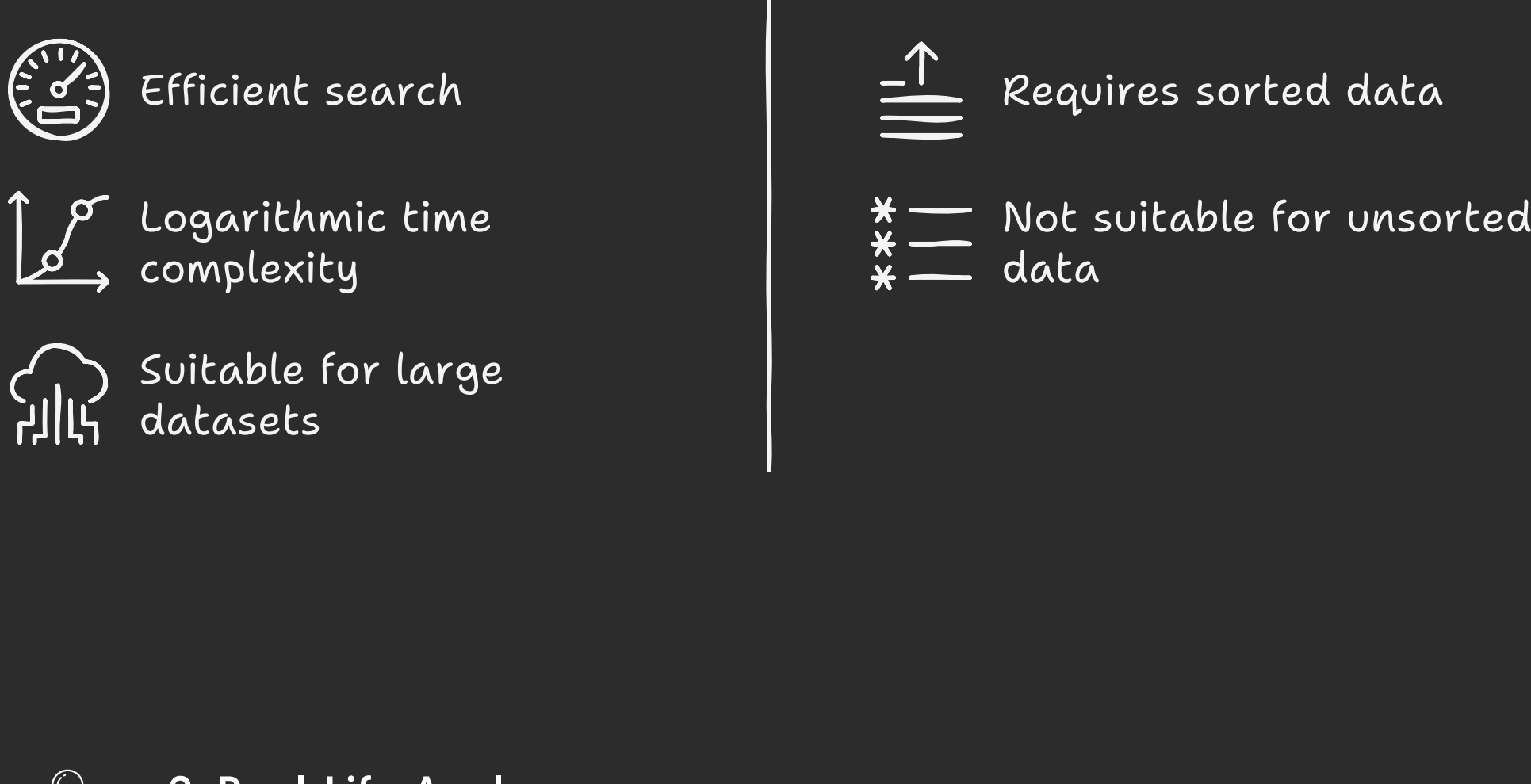
Binary Search

1. Introduction

Binary Search is a classic **search algorithm** used to efficiently locate a target element in a **sorted search space** by repeatedly dividing the search interval in half.

🔗 **Important Prerequisite:** The search space must be sorted.

Binary Search Algorithm



2. Real-Life Analogy

Problem: You need to find the word "raj" in a dictionary.

Method 1: Linear Search

Check each page one-by-one from the beginning to the end.

- ✓ Works always
- ✗ Time-consuming
- Equivalent to **O(N)** time complexity

Method 2: Binary Search

Use the **sorted nature** of the dictionary.

Steps:

- Open the dictionary at the middle.
- Suppose the opened page starts with 's'.
- Since 'raj' comes before 's', search the **left half** only.
- Repeat the above logic recursively until the word is found.

⚠️ **Key Insight:** Binary Search drastically reduces the search space using a divide-and-conquer approach.

3. Conceptual Understanding

- Binary Search is **efficient** and requires the input to be **sorted**.
- At every step:
 - Find the **middle** of the current search space.
 - Decide to move **left** or **right**, depending on the comparison.

4. Problem Statement

Given: A sorted array of integers and a target value. **Goal:** Find the index of the target in the array.

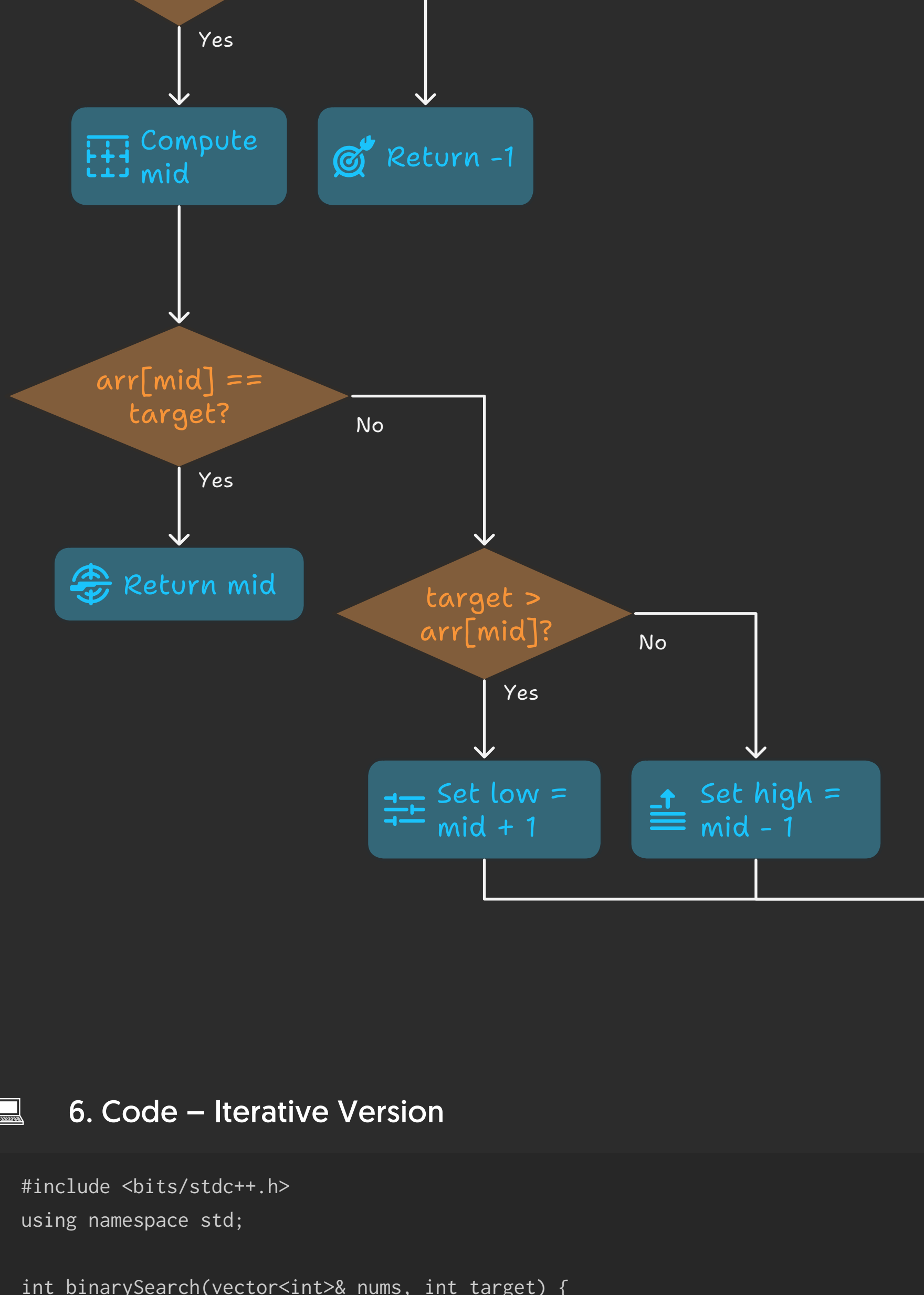
```
Input: nums = [3, 4, 6, 7, 9, 12, 16, 17], target = 6
Output: Index = 2
```

5. Iterative Binary Search – Algorithm

Step-by-Step

- Initialize:
 - low = 0
 - high = n - 1
- Loop until low <= high:
 - Compute mid = (low + high) // 2
 - If arr[mid] == target: return mid
 - If target > arr[mid]: set low = mid + 1
 - If target < arr[mid]: set high = mid - 1
- If not found: return -1

Iterative Binary Search Algorithm



6. Code – Iterative Version

```
#include <bits/stdc++.h>
using namespace std;

int binarySearch(vector<int>& nums, int target) {
    int low = 0, high = nums.size() - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (nums[mid] == target) return mid;
        else if (target > nums[mid]) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    vector<int> a = {3, 4, 6, 7, 9, 12, 16, 17};
    int target = 6;
    int ind = binarySearch(a, target);
    if (ind == -1) cout << "The target is not present." << endl;
    else cout << "The target is at index: " << ind << endl;
    return 0;
}
```

7. Time Complexity Analysis

Each time, the array is **halved**. Let's analyze it:

Let array size = N

At each step:

- New size = N / 2 → N / 4 → N / 8 → ... → 1

We stop when:

$$N / 2^x = 1 \Rightarrow 2^x = N \Rightarrow x = \log_2(N)$$

Final Time Complexity: O(log₂ N)

(Same for iterative and recursive versions)

8. Important Edge Case – Overflow

Using mid = (low + high) / 2 can cause **integer overflow** in some languages (e.g., Java, C++) if low + high exceeds integer limit.

💡 **Safe formula:**

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2;$$

9. Recursive Binary Search – Algorithm

Approach / Intuition

- This approach applies the same logic as iterative binary search.
- At each of looping, it uses **function recursion** to reduce the search space.
- At each recursive call, we adjust either the low or high pointer.

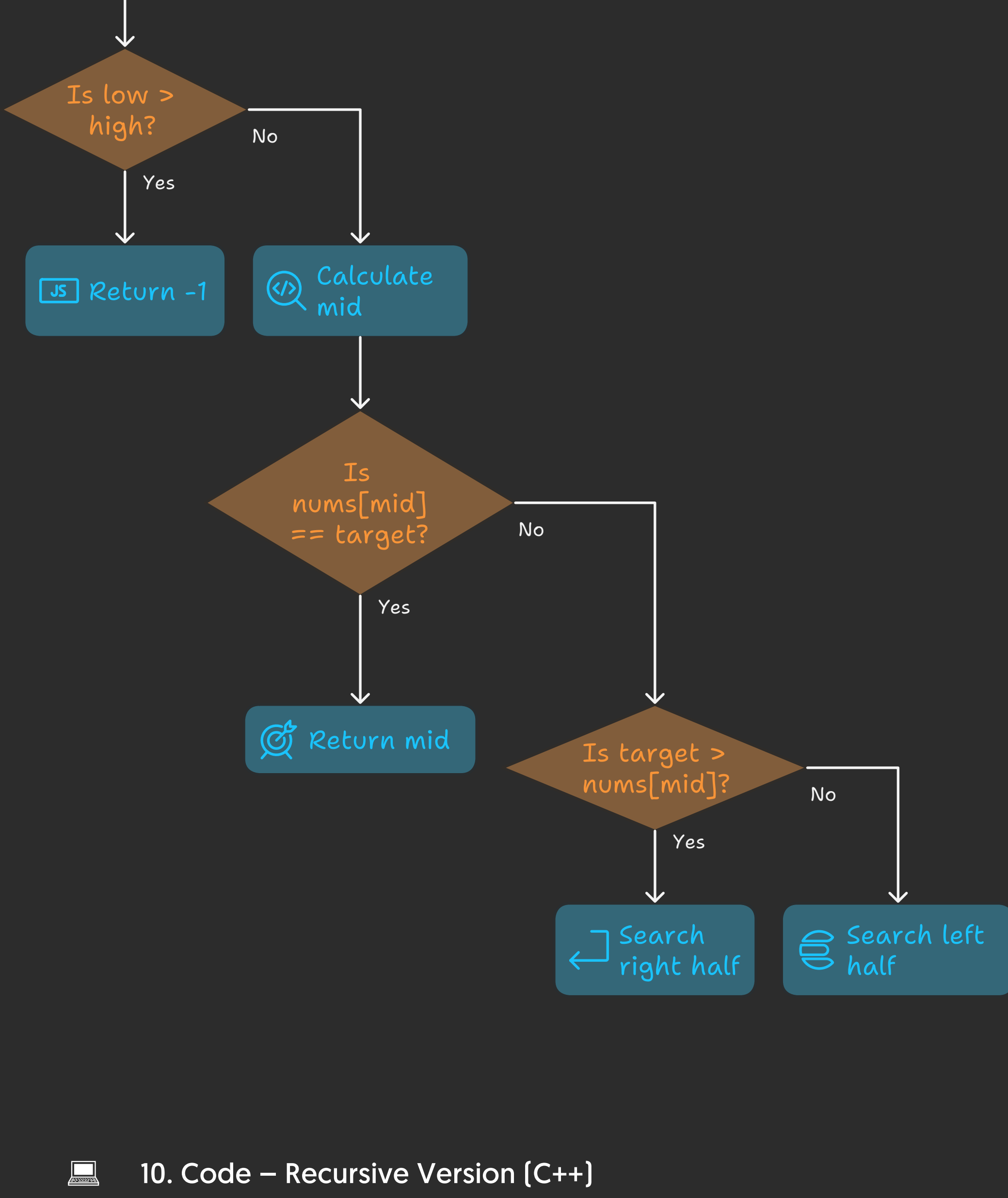
Step-by-Step Algorithm

- Base Case:** If low > high, return -1 [i.e., the search space is invalid → element not found]
- Divide the search space:**

```
int mid = (low + high) / 2;
```

- Compare nums[mid] with target:**
 - If equal: return mid [target found].
 - If target > nums[mid]: Search in right half → binarySearch(nums, mid + 1, high, target)
 - If target < nums[mid]: Search in left half → binarySearch(nums, low, mid - 1, target)

Recursive Binary Search Algorithm



10. Code – Recursive Version [C++]

```
#include <bits/stdc++.h>
using namespace std;

int binarySearch(vector<int>& nums, int low, int high, int target) {
    if (low > high) return -1; // Base case

    int mid = (low + high) / 2; // Divide step

    if (nums[mid] == target) return mid; // Found
    else if (target > nums[mid]) // Search right
        return binarySearch(nums, mid + 1, high, target);
    else // Search left
        return binarySearch(nums, low, mid - 1, target);
}

int search(vector<int>& nums, int target) {
    return binarySearch(nums, 0, nums.size() - 1, target);
}

int main() {
    vector<int> a = {3, 4, 6, 7, 9, 12, 16, 17};
    int target = 6;
    int ind = search(a, target);

    if (ind == -1)
        cout << "The target is not present." << endl;
    else
        cout << "The target is at index: " << ind << endl;

    return 0;
}
```

Output:

```
The target is at index: 2
```

12. Complexity Analysis

Time Complexity

- Same as iterative approach.
- Each recursive call halves the search space.
- So, **O(log₂ N)**

Space Complexity

- Unlike the iterative approach, recursion uses the **function call stack**.
- In the worst case, number of recursive calls = depth of recursion = log₂ N

✓ **Space Complexity = O(log N)**